

jeeves-watcher — Architecture Spec

Version: 0.3.0

Date: 2026-02-22

Status: DEPLOYED — v0.2.5 in production, plugin/skill extension planned

Repo: GitHub: `karmaniverous/jeeves-watcher`

npm: `@karmaniverous/jeeves-watcher`

Channel: `#project-jeeves-watcher` (`C0AGHBKJQDP`)

Overview

jeeves-watcher is a standalone Node.js service that monitors configured filesystem directories and keeps a Qdrant vector store in sync with document changes. It extracts text, generates Gemini embeddings, applies inference rules for metadata classification, and exposes a semantic search API.

Canonical implementation docs live in the repo at `guides/` :

- `guides/architecture.md` — Design principles, data flow, three sources of truth
- `guides/configuration.md` — Full config reference (Zod-validated)
- `guides/inference-rules.md` — Rules engine with JSON Schema matching + JsonMap transforms
- `guides/api-reference.md` — HTTP endpoints (`/search` , `/metadata` , `/reindex` , `/config-reindex` , `/rebuild-metadata` , `/status`)
- `guides/deployment.md` — Production setup (NSSM, systemd)
- `guides/cli-reference.md` — CLI commands

This document tracks:

1. Jeeves-specific deployment configuration and integration points
 2. The OpenClaw plugin and skill architecture (planned for v0.3.0)
-

Production Deployment

Service Topology

Component	Port	Data Location
jeeves-watcher	3458	Config: J:\config\jeeves-watcher.config.json
Qdrant	6333	Data: J:\state\qdrant\
Logs	—	J:\state\watcher\
Metadata store	—	.jeeves-watcher/ sidecars adjacent to watched files

Both run as NSSM Windows services. Target state is Docker Compose on Linux ARM.

Current Scale

- 10,498 points in Qdrant collection jeeves_archive (3072-dim, Cosine)
- 9 domains indexed: email, slack, x, meetings, github, codebase, projects, jira, global
- 21 watch paths covering J:\domains\ and J:\jeeves\
- 25 inference rules for domain classification and metadata extraction

Watch Paths

```
j:/domains/email/cache/**/*.json
j:/domains/email/archive/**/*.json
j:/domains/email/meta.json, meta-state.json
j:/domains/slack/channels/**/*.json, meta.json, register.json
j:/domains/x/raw/**/*.json, meta.json, register.json
j:/domains/meetings/**/*.{txt,md,json}
j:/domains/github/**/*.{md,json}
j:/domains/codebase/**/*.{md,json}
j:/domains/codebase/repos/**/*.{js,ts,jsx,tsx,py,rs,go,java,rb,sh,yml,yaml,toml,json,md,txt}
j:/domains/projects/**/*.{md,txt,json}
j:/domains/global/**/*.{md,json}
j:/domains/jira/**/*.json
j:/domains/finance/**/*.{json,md}
j:/jeeves/memory/**/*.md, MEMORY.md, scratch/**/*.md
```

Ignored: node_modules , .git , .domain , desktop.ini , Thumbs.db , *.mmd , *.pum1 , images, package-lock.json , dist/ .

Inference Rules

Domain classification from file paths:

Path Pattern	Domain	Notes
j:/domains/email/archive/**	email	Message bodies
j:/domains/email/cache/**	email	Thread metadata

Path Pattern	Domain	Notes
j:/domains/email/meta*	meta_email	Email meta (mutable)
j:/domains/slack/channels/**	slack	Messages
j:/domains/slack/meta*, register*	meta_slack	Slack meta
j:/domains/x/raw/**	x	Posts
j:/domains/x/meta*, register*	meta_x	X meta
j:/domains/meetings/**/transcript*	meetings	Transcripts
j:/domains/meetings/**/meeting.json	meetings	Manifests
j:/domains/meetings/meta*, register*	meta_meetings	Meetings meta
j:/domains/github/**	github	Issues, repo indexes
j:/domains/codebase/repos/**	codebase	Repo snapshots
j:/domains/codebase/meta*	meta_codebase	Code meta
j:/domains/projects/**	projects	Project docs
j:/domains/global/**	meta_global	Cross-domain synthesis
j:/domains/jira/**	jira	Sprint data
j:/domains/finance/**	finance	Financial data
j:/jeeves/**	workspace	Agent memory, scratch

Metadata extraction rules:

Source	Target Field	Mechanism
Markdown frontmatter title	title	Template: \${frontmatter.title}
Markdown frontmatter author	author	Template: \${frontmatter.author}
JSON field subject	title	Template: \${json.subject}
JSON field participants (array)	participants	Template: \${json.participants}

All metadata fields are filterable in Qdrant search via filter objects.

Embedding Config

- **Provider:** Gemini (`gemini-embedding-001`)
- **Dimensions:** 3072 (Matryoshka to 1536/768)
- **Chunk size:** 1000 tokens, 200 overlap
- **Rate limit:** 1000/min, concurrency 5
- **Cost:** ~~\$0.15/1M tokens~~ (\$2.74/year ongoing)
- **API key:** `${GOOGLE_API_KEY}` (env var substitution in config)

Three Sources of Truth

Source	Owns	Mutability
Filesystem	Content (files)	External processes write
Metadata store (<code>.meta.json</code> sidecars)	Enrichment metadata	<code>POST /metadata</code> or inference rules
Qdrant	Derived vector index	Fully rebuildable from filesystem + metadata

Integration with Data Flow v7.0.0

jeeves-watcher replaces per-workflow embedding in n8n:

1. **n8n workflows** run domain scripts that write files to `J:\domains\`
2. **jeeves-watcher** detects changes, extracts text, embeds, upserts to Qdrant
3. **n8n meta workflows** call `POST http://localhost:3458/search` for semantic context (back-propagation)
4. **Meta output** written to filesystem → watcher re-embeds (semantic back-propagation loop)

n8n never touches Qdrant directly. The watcher is the sole index writer.

OpenClaw Plugin Architecture (v0.3.0)

Motivation

jeeves-watcher's semantic search is most valuable when an AI agent can use it. Currently, agents must shell out to `curl` or use `exec` to query the API. An OpenClaw plugin makes the watcher a first-class agent capability: a native tool with schema-driven parameters, plus a skill that teaches the agent how to construct effective searches.

Plugin Structure

```
jeeves-watcher/  
└─ src/                # Existing service source
```

```

├─ plugin/                                # OpenClaw plugin (new)
|   ├─ openclaw.plugin.json              # Plugin manifest
|   └─ index.ts                          # Plugin entry point (tool registration)
|       └─ skill/                        # Bundled skill
|           └─ SKILL.md                  # Agent instructions for search
├─ guides/                              # Existing documentation
├─ dist/                                # Built output
|   └─ plugin/                           # Compiled plugin
|       ├─ openclaw.plugin.json
|       └─ index.js
|           └─ skill/
|               └─ SKILL.md
└─ ...                                  # Existing service dist
└─ package.json                          # Updated with openclaw.extensions

```

Plugin Manifest (`openclaw.plugin.json`)

```

{
  "id": "jeeves-watcher",
  "name": "Jeeves Watcher",
  "description": "Semantic search across the Jeeves document archive via jeeves-watcher.",
  "version": "0.3.0",
  "skills": ["skill"],
  "configSchema": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "apiUrl": {
        "type": "string",
        "description": "jeeves-watcher API base URL",
        "default": "http://127.0.0.1:3458"
      }
    }
  },
  "uiHints": {
    "apiUrl": {
      "label": "Watcher API URL",
      "placeholder": "http://127.0.0.1:3458"
    }
  }
}

```

Agent Tool: `watcher\_search`

The plugin registers a single agent tool that wraps the watcher's search API:

Tool name: `watcher_search`

Parameters:

Parameter	Type	Required	Description
<code>query</code>	string	yes	Natural language search query

Parameter	Type	Required	Description
limit	number	no	Max results (default: 10)
domain	string	no	Filter by domain (sugar for Qdrant filter)
filter	object	no	Raw Qdrant filter for advanced queries

Returns: Array of search results with `score`, `file_path`, `chunk_text`, `domain`, `title`, and other metadata fields.

Implementation: The tool makes an HTTP POST to the configured `apiUrl + "/search"`, constructing the Qdrant filter from the `domain` shorthand or passing through the raw `filter` object.

```
// Simplified tool implementation
async function watcherSearch(params: {
  query: string;
  limit?: number;
  domain?: string;
  filter?: object;
}) {
  const body: any = { query: params.query, limit: params.limit ?? 10 };

  if (params.domain) {
    body.filter = {
      must: [{ key: "domain", match: { value: params.domain } }],
    };
  } else if (params.filter) {
    body.filter = params.filter;
  }

  const res = await fetch(`${apiUrl}/search`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body),
  });

  return res.json();
}
```

Agent Tool: `watcher\_enrich`

Enrich a file's metadata without re-embedding (wraps `POST /metadata`):

Parameter	Type	Required	Description
path	string	yes	File path (must be indexed or will be on next event)
metadata	object	yes	Metadata to merge (new values override existing)

Use cases: labeling documents after analysis, adding `reviewed: true` , tagging with project names, setting priority. Wraps the existing `jeeves-watcher enrich` CLI command / `POST /metadata` API.

Agent Tool: `watcher\_status`

Service health and schema discovery:

Parameter	Type	Required	Description
(none)	—	—	Returns status, uptime, and available metadata fields

Schema discovery: The `/status` endpoint will be extended (see \$API Extensions below) to return the set of metadata fields present in the collection, enabling the skill to guide searches dynamically rather than hardcoding field names.

Skill: `SKILL.md`

The bundled skill teaches the agent how to use the tools effectively:

1. **Tool reference** — parameters, return schema, examples
2. **Domain reference** — what content lives in each domain, what metadata fields are available
3. **Search construction guide** — when to use `domain` filter vs. free-text, score interpretation, chunk grouping
4. **Advanced filtering** — Qdrant filter syntax for `must` , `must_not` , `should` , `match.value` vs. `match.text`
5. **Workflow patterns** — "find then read" (search → read file), "find and summarize" (search → synthesize from chunks)

The skill is deployment-aware via the `watcher_status` tool: it instructs the agent to call status first to discover available domains and metadata fields, then construct queries accordingly.

Skill frontmatter:

```
---
name: jeeves-watcher
description: >
  Semantic search across the document archive using jeeves-watcher.
  Use when you need to find documents, recall context, search across domains
  (email, meetings, projects, slack, github, codebase, jira, x), or answer
  questions that require looking up indexed content. Also covers metadata
  enrichment, reindexing, and service management.
---
```

Installation

```
# As OpenClaw plugin
openclaw plugins install @karmaniverous/jeeves-watcher
```

```
# Configure
# In openclaw config: plugins.entries.jeeves-watcher.config.apiUrl = "http://127.0.0.1:3458"
```

The `package.json` declares the plugin entry point:

```
{
  "openclaw": {
    "extensions": [".dist/plugin/index.js"]
  }
}
```

API Extensions (v0.3.0)

Extended `/status`` Response

To support skill-driven schema discovery, the `/status` endpoint returns available metadata fields:

```
{
  "status": "ok",
  "uptime": 86400,
  "collection": {
    "name": "jeeves_archive",
    "pointCount": 10498,
    "dimensions": 3072
  },
  "domains": ["email", "slack", "x", "meetings", "github", "codebase", "projects", "jira", "finance",
"meta_global", "workspace"],
  "metadataFields": ["domain", "title", "author", "participants"]
}
```

This makes the plugin self-describing: the skill instructs the agent to call `watcher_status` to discover what's available, rather than hardcoding domain lists.

Filter Pass-Through on `/search``

The search endpoint already accepts an optional `filter` parameter (Qdrant filter object) passed through to the vector query. This is undocumented in the current guides and should be formalized:

```
interface SearchRequest {
  query: string;
  limit?: number;
  filter?: QdrantFilter; // Pass-through to Qdrant search
}
```


Design Decisions

Decision	Rationale
Plugin ships with service package	Single <code>npm install</code> gets service + CLI + plugin + skill
<code>domain</code> parameter as filter sugar	90% of filtered searches are by domain; avoid forcing raw Qdrant syntax
Schema discovery via <code>/status</code>	Skill stays generic; deployment-specific metadata discovered at runtime
Skill references tools, not curl	Agent uses native <code>watcher_search</code> tool, not <code>exec</code> + curl
Filter pass-through for power users	Full Qdrant filter syntax when <code>domain</code> shorthand isn't enough
Plugin config is just <code>apiUrl</code>	Watcher's own config handles everything else; plugin only needs to find the API

Known Issues

Issue	Status	Notes
#15	Open	Shebang line causes ESM parse error on Windows Node v24
Filter param undocumented	Open	<code>/search</code> accepts <code>filter</code> but guides don't mention it
<code>/status</code> lacks schema info	Planned	Need collection stats + metadata field enumeration

Roadmap

v0.2.x (current)

- ✓ Core service: watch, extract, embed, upsert
- ✓ Inference rules engine with JSON Schema + glob matching
- ✓ Metadata enrichment API (`POST /metadata`)
- ✓ Semantic search API (`POST /search`) with filter pass-through
- ✓ Config-reindex (rules scope and full scope)
- ✓ `.gitignore` support (v0.2.5)
- ✓ BOM stripping and graceful error handling (v0.2.4)
- ✓ Env var substitution in config (v0.2.2)
- ✓ Deployed to production: 10,498 points across 9 domains

v0.3.0 (next)

- ☐ OpenClaw plugin manifest + entry point
- ☐ `watcher_search` agent tool
- ☐ `watcher_enrich` agent tool
- ☐ `watcher_status` agent tool with schema discovery
- ☐ Bundled skill (`SKILL.md`) with search construction guide
- ☐ Extended `/status` with collection stats + metadata fields + domain list
- ☐ Formalize `filter` parameter in `/search` docs
- ☐ Fix shebang issue (#15)

Future

- ☐ PDF/DOCX text extraction (currently JSON/Markdown/plaintext only)
- ☐ Streaming reindex (non-blocking `POST /reindex`)
- ☐ WebSocket file event stream for real-time consumers
- ☐ Docker image for containerized deployment